

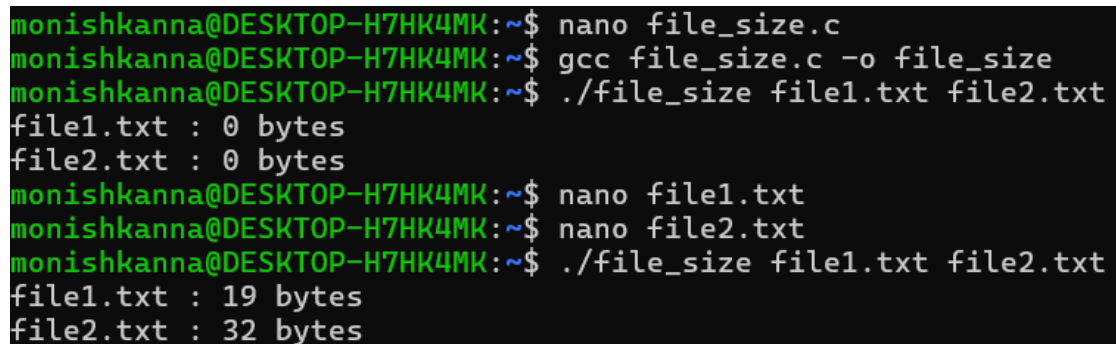
Command Line Arguments of a Process

1. Write a C program that accepts multiple filenames as command line arguments and prints the size of each file.

CODE:

```
#include <stdio.h>
#include <sys/stat.h>
int main(int argc, char *argv[]) {
    struct stat st;
    for(int i=1; i<argc; i++) {
        if(stat(argv[i], &st) == 0)
            printf("%s : %ld bytes\n", argv[i], st.st_size);
        else
            perror("stat failed");
    }
    return 0;
}
```

OUTPUT:



```
monishkanna@DESKTOP-H7HK4MK:~$ nano file_size.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc file_size.c -o file_size
monishkanna@DESKTOP-H7HK4MK:~$ ./file_size file1.txt file2.txt
file1.txt : 0 bytes
file2.txt : 0 bytes
monishkanna@DESKTOP-H7HK4MK:~$ nano file1.txt
monishkanna@DESKTOP-H7HK4MK:~$ nano file2.txt
monishkanna@DESKTOP-H7HK4MK:~$ ./file_size file1.txt file2.txt
file1.txt : 19 bytes
file2.txt : 32 bytes
```

2. Implement a program that parses command line arguments to perform different operations (e.g., add, subtract, multiply) on two integers.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char *argv[]) {
    if(argc != 4){
        printf("Usage: ./calc add 5 3\n");
        return 1;
    }
    int a = atoi(argv[2]);
    int b = atoi(argv[3]);
    if(strcmp(argv[1], "add") == 0)
        printf("Result: %d\n", a+b);
    else if(strcmp(argv[1], "sub") == 0)
        printf("Result: %d\n", a-b);
    else if(strcmp(argv[1], "mul") == 0)
        printf("Result: %d\n", a*b);
    else
        printf("Invalid operation\n");
    return 0;
}
```

OUTPUT:

```
monishkanna@DESKTOP-H7HK4MK:~$ nano calc.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc calc.c -o calc
monishkanna@DESKTOP-H7HK4MK:~$ ./calc add 10 5
Result: 15
```

Environment of a Process

1. Write a C program that prints all environment variables of the current process.

CODE:

```
#include <stdio.h>
extern char **environ;
int main() {
    for(int i=0; environ[i]!=NULL; i++)
        printf("%s\n", environ[i]);
    return 0;
}
```

OUTPUT:

```
monishkanna@DESKTOP-H7HK4MK:~$ ./print_env
SHELL=/bin/bash
WSL2_GUI_APPS_ENABLED=1
WSL_DISTRO_NAME=Ubuntu
NAME=DESKTOP-H7HK4MK
PWD=/home/monishkanna
LOGNAME=monishkanna
MOTD_SHOWN=update-motd
HOME=/home/monishkanna
LANG=C.UTF-8
WSL_INTEROP=/run/WSL/323_interop
```

2. Implement a program that sets a custom environment variable and then retrieves its value using getenv().

CODE:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    setenv("MYVAR", "HelloWorld", 1);
    char *value = getenv("MYVAR");
    printf("MYVAR = %s\n", value);
    return 0;
}
```

OUTPUT:

```
monishkanna@DESKTOP-H7HK4MK:~$ nano set_env.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc set_env.c -o set_env
./set_env
MYVAR = HelloWorld
```

fork

1. Write a C program that creates a child process using fork() and prints the process IDs of both parent and child.

CODE:

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid = fork();
    if(pid == 0)
        printf("Child PID: %d, Parent PID: %d\n", getpid(), getppid());
    else
        printf("Parent PID: %d, Child PID: %d\n", getpid(), pid);
    return 0;
}
```

OUTPUT:

```
monishkanna@DESKTOP-H7HK4MK:~$ nano fork_pid.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc fork_pid.c -o fork_pid
monishkanna@DESKTOP-H7HK4MK:~$ ./fork_pid
Parent PID: 609, Child PID: 610
Child PID: 610, Parent PID: 323
```

2. Implement a program where the parent and child process each write to a shared file after fork(). Analyze the output.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
int main() {
    int fd = open("fork.txt", O_CREAT | O_WRONLY | O_APPEND, 0666);
    if(fork() == 0){
        write(fd, "Child\n", 6);
    } else {
        write(fd, "Parent\n", 7);
        wait(NULL);
    }
    close(fd);
    return 0;
}
```

OUTPUT:

```
monishkanna@DESKTOP-H7HK4MK:~$ nano fork_write.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc fork_write.c -o fork_write
monishkanna@DESKTOP-H7HK4MK:~$ ./fork_write
monishkanna@DESKTOP-H7HK4MK:~$ cat fork.txt
Parent
Child
```

exec

1. Write a C program that uses execvp() to replace the current process image with the ls command.

CODE:

```
#include <stdio.h>
#include <unistd.h>
int main() {
```

```

char *args[] = {"ls", "-l", NULL};
execvp("ls", args);
return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano exec_ls.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc exec_ls.c -o exec_ls
./exec_ls
total 844
-rwxr-xr-x 1 monishkanna monishkanna 16048 Mar 1 17:37 a.out
-rw-r--r-- 1 monishkanna monishkanna 25 Mar 4 16:34 append.txt
-rwxr-xr-x 1 monishkanna monishkanna 16048 Mar 2 17:02 atomic
-rw-r--r-- 1 monishkanna monishkanna 216 Mar 2 17:02 atomic.c
-rw-r--r-- 1 monishkanna monishkanna 13 Mar 2 17:03 atomic.txt
-rwxr-xr-x 1 monishkanna monishkanna 16136 Mar 4 16:34 atomic_append
-rw-r--r-- 1 monishkanna monishkanna 336 Mar 4 16:33 atomic_append.c
-rwxr-xr-x 1 monishkanna monishkanna 16184 Mar 4 14:52 atomic_log
-rw-r--r-- 1 monishkanna monishkanna 362 Mar 4 14:52 atomic_log.c
-rwxr-xr-x 1 monishkanna monishkanna 16424 Mar 4 17:16 atomic_rmw
-rw-r--r-- 1 monishkanna monishkanna 975 Mar 4 17:16 atomic_rmw.c
-rwxr-xr-x 1 monishkanna monishkanna 16088 Mar 5 07:10 calc
-rw-r--r-- 1 monishkanna monishkanna 523 Mar 5 07:10 calc.c
-rwxr-xr-x 1 monishkanna monishkanna 16048 Mar 1 17:14 chmod_sys
-rw-r--r-- 1 monishkanna monishkanna 198 Mar 1 17:10 chmod_sys.c
-rwxr-xr-x 1 monishkanna monishkanna 16216 Mar 1 17:02 copy
-rw-r--r-- 1 monishkanna monishkanna 494 Mar 1 17:01 copy.c
-rw-r--r-- 1 monishkanna monishkanna 14 Mar 1 17:02 copy.txt
-rw-r--r-- 1 monishkanna monishkanna 1 Mar 4 17:16 counter.txt
-rwxr-xr-x 1 monishkanna monishkanna 16088 Mar 1 16:52 create

```

2. Implement a program that forks a child and uses `execlp()` to run a different program, passing command line arguments.

CODE:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {

    if(fork()==0){
        execlp("echo", "echo", "Hello from Child", NULL);
    } else {
        wait(NULL);
        printf("Parent Done\n");
    }
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano fork_exec.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc fork_exec.c -o fork_exec
monishkanna@DESKTOP-H7HK4MK:~$ ./fork_exec
Hello from Child
Parent Done

```

wait and waitpid

1. Write a C program that creates multiple child processes and uses `wait()` to collect their exit statuses.

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int i;
    pid_t pid;
    for(i=0;i<3;i++)
    {
        pid=fork();
        if(pid==0)
        {
            printf("Child %d PID: %d\n",i+1,getpid());
            exit(i+1);
        }
    }
    for(i=0;i<3;i++)
    {
        int status;
        pid_t child=wait(&status);
        printf("Parent collected child %d exit status %d\n",
            child,WEXITSTATUS(status));
    }
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano wait_example.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc wait_example.c -o wait_example
monishkanna@DESKTOP-H7HK4MK:~$ ./wait_example
Child 1 PID: 736
Child 2 PID: 737
Child 3 PID: 738
Parent collected child 736 exit status 1
Parent collected child 737 exit status 2
Parent collected child 738 exit status 3

```

2. Implement a program that uses waitpid() to wait for a specific child process to finish and prints its exit code.

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    int status;
    pid=fork();
    if(pid==0)
    {
        printf("Child process running PID=%d\n",getpid());
        sleep(2);
    }
}

```

```

        exit(5);
    }
    else
    {
        printf("Parent waiting for child %d\n",pid);
        waitpid(pid,&status,0);
        printf("Child exited with status %d\n",WEXITSTATUS(status));
    }
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano waitpid_example.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc waitpid_example.c -o waitpid_example
monishkanna@DESKTOP-H7HK4MK:~$ ./waitpid_example
Parent waiting for child 772
Child process running PID=772
Child exited with status 5

```

Orphan Process and Zombie Process

1. Write a C program that creates an orphan process and demonstrates its adoption by the init process.

CODE:

```

#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        sleep(5);
        printf("Child Process ID: %d\n", getpid());
        printf("Child's Parent PID after parent exit: %d\n", getppid());
    }
    else
    {
        printf("Parent Process ID: %d\n", getpid());
        printf("Parent exiting...\n");
    }
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano orphan.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc orphan.c -o orphan
monishkanna@DESKTOP-H7HK4MK:~$ ./orphan
Parent Process ID: 612
Parent exiting...
monishkanna@DESKTOP-H7HK4MK:~$ Child Process ID: 613
Child's Parent PID after parent exit: 312

```

2. Implement a program that creates a zombie process and shows how it appears in the process table until the parent calls wait().

CODE:

```

#include <stdio.h>
#include <unistd.h>

```

```

int main()
{
    pid_t pid = fork();
    if(pid == 0)
    {
        printf("Child Process ID: %d\n", getpid());
        printf("Child exiting...\n");
    }
    else
    {
        printf("Parent Process ID: %d\n", getpid());
        sleep(10); // Parent not calling wait()
        printf("Parent woke up after sleep\n");
    }
    return 0;
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano zombie.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc zombie.c -o zombie
monishkanna@DESKTOP-H7HK4MK:~$ ./zombie
Parent Process ID: 626
Child Process ID: 627
Child exiting...
Parent woke up after sleep

```

Signals

1. Write a C program that handles SIGINT (Ctrl+C) and prints a message before exiting.

CODE:

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>
void handler(int sig){
    printf("\nCaught SIGINT\n");
}
int main(){
    signal(SIGINT,handler);
    while(1);
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano sigint.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sigint.c -o sigint
monishkanna@DESKTOP-H7HK4MK:~$ ./sigint
^C
Caught SIGINT

```

2. Implement a program that sends SIGUSR1 to another process and handles it with a custom signal handler.

CODE:

```

#include <stdio.h>
#include <signal.h>

```

```

#include <unistd.h>
void handler(int sig){
    printf("SIGUSR1 received\n");
}
int main(){
    signal(SIGUSR1,handler);
    if(fork()==0){
        sleep(1);
        kill(getppid(),SIGUSR1);
    }
    while(1);
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano sigusr1.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sigusr1.c -o sigusr1
monishkanna@DESKTOP-H7HK4MK:~$ ./sigusr1
SIGUSR1 received
^C

```

Signal Ignore, Signal Default

1. Write a C program that ignores SIGINT and demonstrates that pressing Ctrl+C does not terminate the process.

CODE:

```

#include <signal.h>
#include <unistd.h>
int main(){
    signal(SIGINT,SIG_IGN);
    while(1);
}

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano ignore.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc ignore.c -o ignore
monishkanna@DESKTOP-H7HK4MK:~$ ./ignore
^C
^Z
[2]+  Stopped                  ./ignore

```

2. Implement a program that sets the signal handler for SIGTERM to default, then sends SIGTERM to itself and observes the result.

CODE:

```

#include <signal.h>
#include <unistd.h>
int main(){
    signal(SIGTERM,SIG_DFL);
    kill(getpid(),SIGTERM);
    return 0; }

```

OUTPUT:

```

monishkanna@DESKTOP-H7HK4MK:~$ nano sigterm.c
monishkanna@DESKTOP-H7HK4MK:~$ gcc sigterm.c -o sigterm
monishkanna@DESKTOP-H7HK4MK:~$ ./sigterm
Terminated

```